

Roteiro

Roteiro do Trabalho Prático – Sistema de Biblioteca

0. Definir o escopo do sistema

0.1 Tema

Sistema de gerenciamento de biblioteca, com usuários comuns e administradores.

0.2 Entidades principais

As três entidades principais exigidas pelo trabalho serão:

- `Livro`;
- `Usuario`;
- `Emprestimo`.

0.3 Perfis de acesso

Papéis mínimos:

- `USER`;
- `ADMIN`.

Permissões gerais:

- usuários comuns acessam o acervo, o próprio perfil e os próprios empréstimos;
- administradores gerenciam livros, usuários e empréstimos;
- somente administradores podem atribuir o papel `ADMIN`;
- usuários comuns nunca podem alterar seu próprio papel.

0.4 Pendências

- quantos livros um usuário pode pegar simultaneamente;
- duração padrão do empréstimo;
- valor diário da multa;
- quando a multa começa a ser calculada;
- se atraso, multa pendente ou ambos bloqueiam novos empréstimos;
- se o pagamento de multa será implementado;
- se a reserva de livro fará parte do sistema;
- se o usuário poderá excluir sua conta com empréstimos ativos;
- se empréstimos poderão ser excluídos ou somente cancelados;
- se a notificação será realmente enviada por e-mail ou apenas registrada pelo sistema.

1. Desenhar a arquitetura

Cada microserviço deve possuir responsabilidade única, banco de dados próprio, ou schema independente, e API REST própria, conforme exigido pelo trabalho.

1.1 Quais entidades existirão?

Principais

- `Livro`;
- `Usuario`;
- `Emprestimo`.

Cada entidade pertence ao domínio de um único microserviço.

Um microserviço não deve mapear, com JPA, entidades pertencentes ao banco de outro microserviço. Por exemplo, `Emprestimo` não deverá possuir:

```
@ManyToOne
private Usuario usuario;
```

porque `Usuario` pertence ao `usuarios-ms`.

Nesse caso, o empréstimo armazenará apenas os identificadores externos:

```
usuarioId
```

livroId

Os dados complementares serão obtidos pela API do outro serviço.

1.2 Quais microsserviços existirão, o que farão e com quem se comunicarão?

acervo-ms

Responsável por:

- cadastrar livros;
- listar e consultar livros;
- atualizar informações dos livros;
- excluir livros;
- pesquisar livros por título, autor ou ISBN;
- controlar a quantidade disponível;
- informar se um livro está disponível;
- reduzir o estoque quando houver empréstimo;
- aumentar o estoque quando houver devolução.

Entidade principal:

- `Livro`.

Comunicação:

- recebe chamadas do `emprestimos-ms` por OpenFeign.

usuarios-ms

Responsável por:

- cadastro de usuários;
- autenticação;
- geração de JWT (token de autenticação);
- consulta e atualização dos dados cadastrais;
- gestão de papéis `USER` e `ADMIN`;
- ativação, bloqueio ou exclusão de usuários;
- consulta da situação cadastral do usuário.

Entidade principal:

- `Usuario`.

A autenticação ficará dentro desse microsserviço.

Comunicação:

- recebe chamadas do `emprestimos-ms` para confirmar a existência e a situação do usuário;
- publica evento de usuário cadastrado para o RabbitMQ;
- poderá publicar eventos de alteração ou exclusão de usuário.

emprestimos-ms

Responsável por:

- registrar empréstimos;
- registrar devoluções;
- consultar empréstimos;
- controlar datas de retirada e devolução;
- identificar empréstimos em atraso;
- calcular multas;
- verificar se o usuário possui empréstimos vencidos;
- verificar se o usuário possui multa pendente;
- coordenar o fluxo entre usuário e acervo;
- publicar eventos relacionados ao empréstimo.

Entidade principal:

- `Emprestimo`.

Comunicação:

- consulta o `usuarios-ms` com OpenFeign;
- consulta e atualiza o `acervo-ms` com OpenFeign;

- publica eventos no RabbitMQ para o `notificacoes-ms`.

`notificacoes-ms`

Responsável por:

- consumir eventos do RabbitMQ;
- criar notificações;
- registrar o resultado do processamento;
- enviar e-mails (opcional);
- evitar que problemas de notificação interrompam o fluxo principal.

Entidade opcional:

- `Notificacao`.

Comunicação:

- consome eventos publicados por `usuarios-ms` e `emprestimos-ms`;
- não participa da resposta síncrona de cadastro, empréstimo ou devolução.

1.3 Autenticação entre frontend e microsserviços

O `usuarios-ms` será responsável pelo login e pela emissão do JWT.

Os demais microsserviços protegidos deverão validar o JWT recebido. Não é recomendável que cada requisição seja enviada ao `usuarios-ms` apenas para verificar se o token é válido.

O fluxo esperado será:

1. o frontend envia e-mail e senha ao `usuarios-ms`;
2. o `usuarios-ms` autentica o usuário;
3. o `usuarios-ms` devolve o JWT;
4. o frontend armazena o token;
5. o Axios envia o token sempre que o usuário acessar um endpoint protegido, como consultar o próprio perfil, consultar os próprios empréstimos, cadastrar um livro ou gerenciar usuários;
6. cada microsserviço protegido valida o token e as permissões.

Obs.: endpoints públicos, como login, cadastro público, consultar acervo, não precisam de autenticação.

1.4 Fonte oficial da multa

- Cada empréstimo possuirá sua própria multa.
- O `emprestimos-ms` será responsável por calcular, armazenar e consultar multas.
- O `usuarios-ms` não armazenará valores de multa, mantendo somente dados cadastrais, autenticação, papéis e situação administrativa.
- O vínculo entre multa e usuário será feito pelo campo `usuarioId` armazenado no empréstimo.
- O valor total devido pelo usuário será obtido pela soma das multas pendentes associadas ao seu identificador.
- Antes de registrar um novo empréstimo, o `emprestimos-ms` verificará em sua própria base se o usuário possui empréstimos atrasados ou multas pendentes.
- O `usuarios-ms` será consultado apenas para confirmar a existência, a identidade e a situação cadastral do usuário.

2. Modelar os dados

Cada microsserviço terá seu próprio schema no banco de dados. Por exemplo:

- `acervo`;
- `usuarios`;
- `emprestimos`;
- `notificacoes`, opcional, caso o serviço persista notificações.

2.1 `acervo-ms`

Entidade `Livro`

Possíveis atributos:

- `id`;
- `titulo`;
- `autor`;
- `isbn`;
- `ativo` (opcional);

- `quantidadeTotal`;
- `quantidadeDisponivel`;
- data de criação (opcional);
- data de atualização (opcional).

Regras iniciais:

- título obrigatório;
- autor obrigatório;
- ISBN obrigatório e único;
- quantidade não pode ser negativa;
- quantidade disponível não pode ser superior à quantidade total.

2.2 usuarios-ms

Entidade `Usuario`

Possíveis atributos:

- `id`;
- `cpf`;
- `nome`;
- `email`;
- `senha`;
- `role`;
- `ativo` (opcional);
- data de criação (opcional);
- data de atualização (opcional).

Regras iniciais:

- CPF obrigatório e único;
- e-mail obrigatório e único;
- senha armazenada de forma criptografada;
- usuário criado pelo cadastro público recebe `USER`;
- somente `ADMIN` pode criar outro `ADMIN`.

2.3 emprestimos-ms

Entidade `Emprestimo`

Possíveis atributos:

- `id`;
- `usuarioId`;
- `livroId`;
- `dataEmprestimo`;
- `dataPrevistaDevolucao`;
- `dataDevolucao`;
- `status`;
- `valorMulta`;
- `multaPaga`;
- data de criação (opcional);
- data de atualização (opcional).

Possíveis status:

- `ATIVO`;
- `DEVOLVIDO`;
- `ATRASADO`;
- `CANCELADO`.

A situação não deve depender apenas de um campo booleano como `devolvido`, caso o sistema precise representar atraso e cancelamento.

2.4 notificacoes-ms

Entidade `opcional` `Notificacao`

Possíveis atributos:

- `id`;
- `tipo`;
- `destinatario`;
- `assunto`;
- `mensagem`;
- `status`;
- `dataRecebimento`;
- `dataEnvio`;
- `erro`.

2.5 Relacionamentos

Os relacionamentos internos de um mesmo microserviço podem ser definidos com JPA.

Os relacionamentos entre microserviços serão lógicos, por identificadores:

```
Emprestimo.usuarioId -> Usuario.id
Emprestimo.livroId   -> Livro.id
```

Não haverá chave estrangeira entre bancos distintos.

3. Padronizar contratos

3.1 Endpoints-base

Como convenção REST, recomenda-se que os recursos sejam escritos no plural:

- `/api/v1/livros`;
- `/api/v1/usuarios`;
- `/api/v1/emprestimos`;
- `/api/v1/notificacoes`.

3.2 DTOs

Todas as requisições devem enviar ou receber DTOs, nunca entidades.

É recomendável separar os DTOs por finalidade:

- `LivroRequest`;
- `LivroResponse`;
- `UsuarioRequest`;
- `UsuarioResponse`;
- `EmprestimoRequest`;
- `EmprestimoResponse`;
- `LoginRequest`;
- `TokenResponse`.

DTOs com sufixo `Request` representam os dados que a API **recebe**.

DTOs com sufixo `Response` representam os dados que a API **devolve**.

Nas operações de cadastro e atualização, a API receberá um DTO `Request` e poderá retornar um DTO `Response` com os dados do recurso criado ou atualizado.

3.3 ResponseEntity

Para padronizar as respostas, todas as requisições deverão retornar um `ResponseEntity`, isto é, todos os métodos dos controllers deverão retornar um objeto `ResponseEntity`.

Tecnicamente, o Spring também permite devolver diretamente o DTO, mas `ResponseEntity` será útil para controlar:

- código HTTP;
- cabeçalhos;
- corpo da resposta;
- respostas sem conteúdo.

3.4 Status HTTP

Padronizar respostas como:

- `200 OK`: consulta ou atualização;
- `201 Created`: criação;

- `204 No Content`: exclusão ou operação sem corpo;
- `400 Bad Request`: dados inválidos;
- `401 Unauthorized`: ausência ou token inválido;
- `403 Forbidden`: falta de permissão;
- `404 Not Found`: recurso inexistente;
- `409 Conflict`: violação de regra de negócio;
- `500 Internal Server Error`: erro inesperado.

3.5 Formato de erro

Definir um formato comum para todos os microserviços:

```
{
  "timestamp": "2026-07-12T14:30:00",
  "status": 409,
  "erro": "CONFLITO",
  "mensagem": "Livro sem exemplares disponíveis",
  "caminho": "/api/v1/emprestimos"
}
```

Para erros de validação:

```
{
  "timestamp": "2026-07-12T14:30:00",
  "status": 400,
  "erro": "DADOS_INVALIDOS",
  "campos": {
    "titulo": "deve possuir ao menos 3 caracteres",
    "autor": "não pode estar vazio"
  }
}
```

3.9 Eventos RabbitMQ

Definir previamente:

- nome do exchange;
- nomes das filas;
- routing keys;
- formato das mensagens;
- quais serviços publicam;
- quais serviços consomem.

Possíveis eventos:

- `UsuarioCadastradoEvent`;
- `EmprestimoCriadoEvent`;
- `EmprestimoDevolvidoEvent`;
- `EmprestimoProximoVencimentoEvent`;
- `MultaAtualizadaEvent`.

Os serviços de domínio, como `usuarios-ms` e `emprestimos-ms`, publicam os eventos. O `notificacoes-ms` consome esses eventos.

O serviço de notificações não precisa ser responsável pela publicação dos eventos de negócio que originam a notificação.

4. Construir a infraestrutura

4.1 Organização inicial do repositório

Estrutura:

```
biblioteca/
├── backend/
│   ├── usuarios-ms/
│   ├── acervo-ms/
│   ├── emprestimos-ms/
│   └── notificacoes-ms/
├── frontend/
│   └── biblioteca-web/
├── infra/
│   ├── docker-compose.yml
│   └── init.sql
```

```
├── docker/
├── docs/
│   ├── arquitetura/
│   ├── diagramas/
│   └── postman/
├── .env.example
└── README.md
```

4.2 Configuração infraestrutura

O roteiro abaixo é para o sistema operacional Linux.

Etapas preliminares

- Instalar Docker Engine e Docker Compose.
- Ativar o serviço Docker.
- Garantir que os arquivos `docker-compose.yml` e `init.sql` estejam na mesma pasta `infra`, já que o Compose utiliza `./init.sql` como *bind mount* para o PostgreSQL.

Inicialização dos contêineres

O arquivo `docker-compose.yml` contém as definições dos serviços da aplicação, incluindo:

- imagem do PostgreSQL;
- nome do contêiner do banco;
- variáveis de ambiente do banco;
- mapeamento de portas;
- volumes;
- *health check*;
- imagem e configuração do RabbitMQ.

Para inicializar os contêineres, execute:

```
# Caminho da pasta onde o arquivo docker-compose.yml está armazenado
cd "/mnt/Arquivos_D0P6LA/Codes/TRABALHO-PWEB/infra" # Exemplo

# Cria e inicializa os serviços definidos no docker-compose.yml
docker compose up -d
```

O comando acima:

- lê o arquivo `docker-compose.yml` da pasta atual;
- baixa as imagens necessárias, caso ainda não existam;
- cria os volumes nomeados `postgres_data` e `rabbitmq_data`;
- cria a rede `biblioteca_network`;
- cria e inicia os contêineres `biblioteca_postgres` e `biblioteca_rabbitmq`;
- tenta executar automaticamente o arquivo `init.sql` na primeira inicialização do PostgreSQL, por meio do diretório `/docker-entrypoint-initdb.d/init.sql`;
- libera o terminal por estar em modo *detached* (`-d`).

Para verificar se os contêineres estão em execução:

```
docker ps
```

Exemplo de saída esperada

Exemplo de situação **ok**:

CONTAINER ID	IMAGE	NAMES	STATUS
abcd1234	postgres:17	biblioteca_postgres	Up ...
efgh5678	rabbitmq:4-management	biblioteca_rabbitmq	Up ...

Exemplo de situação **não ok**:

- um dos contêineres não aparece no `docker ps`;
- ou o `STATUS` indica reinicialização ou falha em vez de `Up`.

Verificação do banco

O banco de dados `biblioteca` é criado automaticamente pela imagem oficial do PostgreSQL com base na variável de ambiente `POSTGRES_DB=biblioteca`.

Para verificar os schemas criados no banco:

```
docker exec -it biblioteca_postgres psql -U admin -d biblioteca -c '\dn'
```

O comando acima:

- executa um comando dentro do contêiner `biblioteca_postgres`;
- abre o cliente `psql`;
- conecta com o usuário `admin` ao banco `biblioteca`;
- lista os schemas existentes utilizando o comando `\dn`.

Exemplo de saída automática bem-sucedida

Se o `init.sql` tiver sido executado automaticamente com sucesso, a saída esperada será semelhante a:

```
List of schemas
Name | Owner
-----+-----
acervo | admin
emprestimos | admin
notificacoes | admin
public | pg_database_owner
usuarios | admin
(5 rows)
```

Isso indica que os schemas do projeto foram criados corretamente.

Exemplo de saída automática com falha

Se a saída mostrar apenas:

```
List of schemas
Name | Owner
-----+-----
public | pg_database_owner
(1 row)
```

então os schemas do projeto **não** foram criados automaticamente.

Criação manual dos schemas

Se os schemas não tiverem sido criados automaticamente, execute a criação manual diretamente no PostgreSQL:

```
docker exec -it biblioteca_postgres psql -U admin -d biblioteca -c "CREATE SCHEMA IF NOT EXISTS usuarios AUTHORIZATION admin;"
docker exec -it biblioteca_postgres psql -U admin -d biblioteca -c "CREATE SCHEMA IF NOT EXISTS acervo AUTHORIZATION admin;"
docker exec -it biblioteca_postgres psql -U admin -d biblioteca -c "CREATE SCHEMA IF NOT EXISTS emprestimos AUTHORIZATION admin;"
docker exec -it biblioteca_postgres psql -U admin -d biblioteca -c "CREATE SCHEMA IF NOT EXISTS notificacoes AUTHORIZATION admin;"
```

Esses comandos criam os schemas manualmente no banco `biblioteca`, utilizando o usuário `admin`. A cláusula `IF NOT EXISTS` evita erro caso algum schema já exista.

Depois, verifique novamente:

```
docker exec -it biblioteca_postgres psql -U admin -d biblioteca -c '\dn'
```

Observação importante

No ambiente de desenvolvimento, a execução automática do `init.sql` pode falhar caso existam problemas de permissão no `bind mount` do arquivo localizado em `/mnt/Arquivos_D0P6LA/...`, mesmo que o `docker-compose.yml` esteja configurado corretamente. Nessa situação, a criação manual dos schemas é uma alternativa válida para inicializar o ambiente e prosseguir com o desenvolvimento.

4.3 Configurações e segredos

Não versionar:

- senhas reais;
- chave privada;
- segredo JWT;
- credenciais de e-mail.

Criar:

```
.env.example
```


com valores de exemplo e documentar quais variáveis precisam ser configuradas.

4.4 Portas

Definir uma tabela fixa:

Componente	Porta sugerida
<code>usuarios-ms</code>	8081
<code>acervo-ms</code>	8082
<code>emprestimos-ms</code>	8083
<code>notificacoes-ms</code>	8084
Frontend	5173
RabbitMQ	5672
RabbitMQ Management	15672
PostgreSQL	5433

Os números são apenas uma convenção e podem ser alterados.

5. Implementar o backend

O backend utilizará arquitetura em camadas, com Controllers, Services, Repositories, DTOs, mapeamento, tratamento global de exceções e validação, conforme solicitado no enunciado.

5.1 Tecnologias do backend

- **Java 21:** linguagem utilizada no desenvolvimento dos microsserviços.
- **Spring Boot:** criação e configuração dos microsserviços.
- **Spring Web:** criação das APIs REST, controllers, endpoints e tratamento de requisições e respostas HTTP.
- **Spring Data JPA:** persistência das entidades, criação dos repositories e acesso ao PostgreSQL.
- **Bean Validation:** validação dos DTOs de entrada.
- **PostgreSQL:** sistema gerenciador de banco de dados para persistência dos dados dos microsserviços.
- **Spring Security com JWT:** autenticação, autorização, proteção dos endpoints e controle de acesso.
- **OpenFeign:** comunicação síncrona entre microsserviços.
- **RabbitMQ:** comunicação assíncrona entre microsserviços por meio de eventos.
- **Swagger/OpenAPI:** documentação e testes dos endpoints das APIs.

5.2 Camadas mínimas por microsserviço

- entidades;
- DTOs;
- repositories;
- services;
- controllers;
- exceptions;
- mappers;
- configurações;
- segurança, quando aplicável;
- clientes Feign, quando aplicável;
- produtores ou consumidores RabbitMQ, quando aplicável.

Estrutura possível:

```
src/main/java/.../  
├── config/  
├── controller/  
├── dto/  
│   ├── request/  
│   └── response/  
├── entity/  
├── exception/  
├── mapper/  
├── repository/  
├── security/  
├── service/  
├── client/  
└── messaging/
```

Nem todo microsserviço terá todos os pacotes.

5.3 Validações

Tratar regras de validação com Bean Validation nos DTOs, nas operações de cadastro e atualização.

Separar:

- validações estruturais: Bean Validation;
- regras de negócio: Services;
- restrições de persistência: entidades e banco.

Exemplo:

- e-mail vazio: Bean Validation;
- ISBN já cadastrado: Service/Repository;
- usuário com multa tentando emprestar: Service;
- coluna `email` única: restrição do banco.

5.4 Tratamento global de exceções

Tratar exceções globalmente em cada microserviço.

Criar classes específicas, por exemplo:

- `UsuarioNaoEncontradoException`;
- `LivroNaoEncontradoException`;
- `EmprestimoNaoEncontradoException`;
- `LivroIndisponivelException`;
- `UsuarioBloqueadoException`;
- `MultaPendenteException`;
- `OperacaoNaoPermitidaException`;
- `IntegracaoIndisponivelException`.

Cada microserviço deverá possuir seu próprio `@RestControllerAdvice`.

5.5 Papéis mínimos

- `USER`;
- `ADMIN`.

5.7 Fluxos principais

- CRUD de usuários;
- CRUD de livros;
- CRUD de empréstimos;
- login.

5.8 Regras principais

Usuários

- qualquer usuário pode se cadastrar como `USER`;
- qualquer usuário autenticado pode consultar seus próprios dados;
- qualquer usuário autenticado pode atualizar seus próprios dados permitidos;
- qualquer usuário pode excluir seu perfil, desde que as regras de negócio permitam;
- somente `ADMIN` pode criar um usuário com papel `ADMIN`;
- somente `ADMIN` pode consultar, atualizar ou excluir qualquer usuário;
- usuário comum não pode alterar seu próprio papel;
- senha nunca deve ser devolvida pelos DTOs de resposta;
- a exclusão deverá considerar empréstimos ativos e multas pendentes.

Acervo

- qualquer usuário pode consultar o acervo;
- somente `ADMIN` pode cadastrar livros;
- somente `ADMIN` pode atualizar livros;
- somente `ADMIN` pode excluir livros;
- sem estoque, o livro não pode ser emprestado;
- não excluir livro associado a empréstimo ativo;
- ISBN deve ser único;
- quantidade disponível nunca pode ser negativa.

Empréstimos

- qualquer usuário pode consultar seus empréstimos;
- somente `ADMIN` pode cadastrar um empréstimo;
- somente `ADMIN` pode consultar empréstimos de todos os usuários;
- somente `ADMIN` pode alterar manualmente dados de qualquer empréstimo;
- sem estoque, não empresta;
- com atraso ou multa pendente, não empresta;
- devolução atualiza disponibilidade;
- atraso calcula multa;
- empréstimo devolvido não pode ser devolvido novamente;
- livro de empréstimo ativo não deve ser alterado;
- exclusão de empréstimo deverá ser restrita e preservar o histórico.

Notificações

- cadastrar usuário envia notificação;
- emprestar livro envia notificação;
- devolver livro envia notificação;
- empréstimo próximo do prazo de devolução envia notificação;
- atualização da multa envia notificação.

5.9 Comunicação síncrona com OpenFeign

Fluxo de empréstimo:

1. o `emprestimos-ms` recebe a solicitação;
2. identifica o usuário;
3. consulta o `usuarios-ms`;
4. verifica no próprio banco se existem atrasos ou multas;
5. consulta o `acervo-ms`;
6. solicita a retirada de um exemplar;
7. registra o empréstimo;
8. publica o evento no RabbitMQ;
9. devolve a resposta ao cliente.

Fluxo de devolução:

1. o `emprestimos-ms` localiza o empréstimo;
2. valida se ainda está ativo;
3. calcula atraso e multa;
4. registra a devolução;
5. solicita ao `acervo-ms` o retorno do exemplar;
6. publica evento no RabbitMQ;
7. devolve a resposta.

5.10 Comunicação assíncrona com RabbitMQ

Os serviços de domínio publicam os eventos.

O `notificacoes-ms` consome os eventos.

Fluxo:

```

emprestimos-ms
|
| publica EmprestimoCriadoEvent
v
RabbitMQ
|
| entrega mensagem
v
notificacoes-ms

```

O envio da notificação não deve impedir que o empréstimo seja concluído.

5.11 Empréstimos próximos do vencimento

Para publicar uma notificação antes do vencimento, será necessário um processo periódico no `emprestimos-ms`.

Ele deverá:

1. localizar empréstimos ativos próximos da data prevista;
2. evitar notificações duplicadas;

3. publicar o evento;
4. deixar o `notificacoes-ms` realizar o envio.

5.13 Ordem recomendada para desenvolver o backend

1. criar os quatro projetos Spring Boot;
2. configurar conexão com os bancos;
3. implementar `usuarios-ms`;
4. implementar autenticação JWT;
5. implementar `acervo-ms`;
6. implementar a estrutura principal de `emprestimos-ms`;
7. definir e implementar os clientes OpenFeign;
8. validar o fluxo completo de empréstimo e devolução;
9. configurar RabbitMQ;
10. implementar publicação de eventos;
11. implementar `notificacoes-ms`;
12. configurar Swagger;
13. revisar validações e exceções;
14. executar testes integrados.

Para evitar que um integrante fique bloqueado pela dependência de outro serviço, os contratos devem ser definidos antes e podem ser usados mocks ou respostas temporárias durante o desenvolvimento.

7. Desenvolver o frontend

O frontend deverá consumir exclusivamente as APIs REST desenvolvidas pela equipe e deverá possuir telas de início, listagem, cadastro, edição, exclusão e consulta.

7.1 Tecnologias do frontend

- **ReactJS:** construção da interface gráfica e criação de componentes reutilizáveis.
- **React Router:** navegação e gerenciamento de rotas da aplicação.
- **Axios:** comunicação HTTP entre o frontend e os microsserviços.
- **React Hooks:** gerenciamento de estado, ciclo de vida dos componentes e lógica reutilizável.
- **Material UI ou Bootstrap:** estilização da interface e utilização de componentes visuais.

7.2 Telas

Tela inicial

- apresentação do sistema;
- acesso ao login;
- acesso ao cadastro;
- acesso ao acervo público, caso permitido.

Tela de login/cadastro

Login:

- e-mail;
- senha;
- exibição de erros;
- armazenamento do JWT;
- redirecionamento conforme o papel.

Cadastro:

- nome;
- CPF;
- e-mail;
- senha;
- confirmação de senha.

Tela de perfil do usuário – opcional

- consultar dados;
- atualizar dados permitidos;
- alterar senha;

- excluir perfil;
- consultar situação.

Tela de acervo

- listar livros;
- pesquisar;
- visualizar disponibilidade;
- consultar detalhes;
- solicitar empréstimo, se essa regra for adotada.

Tela de empréstimos do usuário

- listar empréstimos ativos;
- listar histórico;
- visualizar prazo;
- visualizar atraso;
- visualizar multa;
- registrar ou solicitar devolução, conforme a regra escolhida.

Tela de administrador

- CRUD de livros;
- CRUD de usuários;
- criação de administradores;
- listagem de empréstimos;
- registro e devolução de empréstimos;
- consulta de atrasos;
- consulta de multas;
- consulta de notificações, caso implementada.

7.3 Segurança no frontend

O frontend deverá:

- armazenar e enviar o JWT;
- ocultar funcionalidades conforme o papel;
- proteger rotas administrativas;
- tratar 401 e 403;
- realizar logout quando o token for inválido.

A proteção visual do frontend não substitui a autorização no backend.

7.4 CORS

Como frontend e microsserviços provavelmente utilizarão portas diferentes, será necessário configurar CORS nos serviços Spring Boot.

7.5 Ordem recomendada do frontend

1. criar projeto React;
2. escolher Material UI ou Bootstrap;
3. configurar React Router;
4. configurar Axios;
5. implementar autenticação;
6. implementar rotas protegidas;
7. implementar tela do acervo;
8. implementar tela de empréstimos;
9. implementar área administrativa;
10. integrar mensagens de erro;
11. testar todos os fluxos.

8. Documentar a API com Swagger/OpenAPI

Cada microsserviço terá sua própria documentação Swagger.

Documentar:

- endpoints;

- DTOs;
- parâmetros;
- permissões;
- códigos de resposta;
- possíveis erros;
- exemplos de entrada e saída;
- autenticação JWT.

Embora apareça como uma etapa final no roteiro, a documentação deve ser atualizada durante a implementação de cada endpoint.

Registrar no README o endereço do Swagger de cada microsserviço.

9. Elaborar coleção de testes com Insomnia ou Postman

Criar uma coleção contendo:

- cadastro;
- login;
- CRUD de usuários;
- CRUD de livros;
- CRUD de empréstimos;
- devolução;
- consultas;
- testes de permissões;
- testes de validação;
- testes de conflitos;
- testes de integração.

Utilizar variáveis:

- URL do `usuarios-ms`;
- URL do `acervo-ms`;
- URL do `emprestimos-ms`;
- token do usuário;
- token do administrador;
- IDs criados durante os testes.

Organizar a coleção por microsserviço.

10. Elaborar o README

O README deverá conter, no mínimo:

- descrição do projeto;
- tema;
- funcionalidades;
- arquitetura;
- diagrama dos microsserviços;
- tecnologias;
- estrutura de pastas;
- bancos utilizados;
- comunicação síncrona;
- comunicação assíncrona;
- autenticação;
- instruções de configuração;
- variáveis de ambiente;
- instruções para executar o Docker Compose;
- instruções para executar os microsserviços;
- instruções para executar o frontend;
- URLs do Swagger;
- credenciais de demonstração, sem expor segredos reais;
- divisão das responsabilidades da equipe.

Esses itens correspondem diretamente ao conteúdo de entrega exigido no documento.

Ordem resumida de execução

1. fechar regras e escopo;
2. desenhar arquitetura;
3. definir responsabilidades dos serviços;
4. modelar bancos e entidades;
5. padronizar endpoints, DTOs, erros e eventos;
6. criar repositório e projetos;
7. criar Docker Compose com PostgreSQL e RabbitMQ;
8. implementar `usuarios-ms` e JWT;
9. implementar `acervo-ms`;
10. implementar `emprestimos-ms`;
11. integrar OpenFeign;
12. implementar RabbitMQ;
13. implementar `notificacoes-ms`;
14. documentar com Swagger;
15. testar o backend;
16. desenvolver frontend React;
17. integrar frontend e backend;
18. criar coleção Postman/Insomnia;
19. concluir README;
20. testar ambiente limpo;
21. preparar apresentação;
22. treinar alterações individuais.